

Securing AWS Resources: Managing Access with IAM

Getting Started

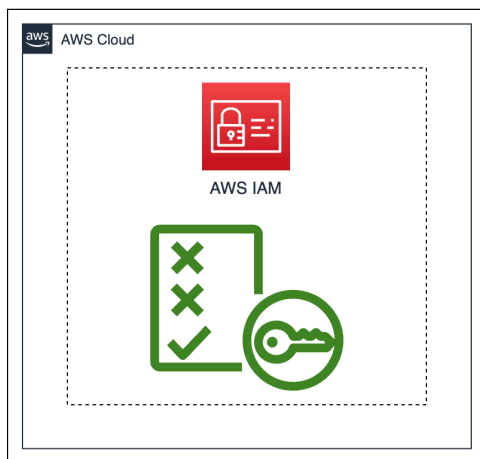
Identity and Access Management (IAM) is a crucial part of any organization's security strategy. It allows controlling access to resources based on the identities of the entities trying to access them. We can grant resource access to the users based on their role and minimize the risk of unauthorized access to sensitive data. This helps organizations with compliance by providing an auditable record of user access to sensitive data and systems.

IAM Identity-Based Policy

An **IAM policy** is a JSON document that allows us to define permissions for an IAM identity or a resource.

Once attached to an identity (users, groups, or roles) and aptly termed an identity-based policy, the policy document allows us to define the actions or operations that the associated identity should be permitted (or denied) to do on the specified resources. Similarly, the policy document, once attached to a resource, and termed as the resource-based policy, allows us to define the access permissions for the particular resource.

In this task, we will focus on identity-based policies. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Architecture diagram

Note: We have a pre-created policy, **IAMLabPolicy**, already there for this lab, and you are neither required nor authorized to create one.

We'll use the pre-created policy in the upcoming tasks. However, for your understanding, the steps to create a policy are listed below:

- Search "IAM" on the AWS console and click "IAM" from the search results. This takes us to the IAM dashboard.
- Click "Policies" under "Access management" from the left sidebar.
- Click the "Create policy" button. This will redirect us to the create policy page.
- Click the "JSON" tab.
- Write the JSON for the IAM policy in the editor window. This JSON can comprise one or more statements where each statement consists of the following components:
 - **Effect:** This specifies if the associated actions are allowed or denied. Its value can either be **Allow** or **Deny**.
 - **Action:** This contains a list of actions to be allowed or denied.
 - **Resource:** This contains a list of resources to perform specified actions.
 - **Condition:** This can be a list of additional restrictions regarding when a policy can be in effect.

The JSON for the pre-created policy, **IAMLabPolicy**, that we'll use in this lab, is given below:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LambdaActions",
      "Effect": "Allow",
      "Action": [
        "lambda:CreateFunction",
        "lambda:GetFunction",
        "lambda:UpdateFunctionCode",
        "lambda:DeleteFunction",
        "lambda:InvokeFunction"
      ]
    }
  ]
}
```

```

    },
    "Resource": [
      "arn:aws:lambda:*:*:function:TestLambdaFunction",
      "arn:aws:lambda:*:*:function:TestLambdaFunction:$LATEST"
    ]
  },
  {
    "Sid": "IAMActions",
    "Effect": "Allow",
    "Action": [
      "iam:CreateRole",
      "iam:DeleteRole",
      "iam:GetRole"
    ],
    "Resource": [
      "arn:aws:iam::*:role/TestLambdaRole"
    ]
  },
  {
    "Sid": "RoleAssumption",
    "Effect": "Allow",
    "Action": "iam:PassRole",
    "Resource": "arn:aws:iam::*:role/TestLambdaRole",
    "Condition": {
      "StringEquals": {
        "iam:PassedToService": [
          "lambda.amazonaws.com"
        ]
      }
    }
  },
  {
    "Sid": "AttachDetachRoles",
    "Effect": "Allow",
    "Action": [
      "iam:DetachRolePolicy",
      "iam:AttachRolePolicy"
    ],
    "Resource": "arn:aws:iam::*:role/TestLambdaRole",
    "Condition": {
      "ArnEquals": {
        "iam:PolicyARN": [
          "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
        ]
      }
    }
  },
  {
    "Sid": "S3Actions",
    "Effect": "Allow",
    "Action": [
      "s3:CreateBucket",
      "s3:GetBucketPolicy",
      "s3:PutEncryptionConfiguration",
      "s3:GetEncryptionConfiguration",
      "s3:GetBucketPublicAccessBlock",
      "s3:PutBucketPublicAccessBlock",
      "s3:PutBucketOwnershipControls",
      "s3:DeleteBucketPolicy",
      "s3:PutBucketPolicy",
      "s3:GetAccountPublicAccessBlock",
      "s3:DeleteBucket",
      "s3:GetBucketPolicyStatus",
      "s3:PutObject",
      "s3:GetObject",
      "s3:DeleteObject"
    ],
    "Resource": [
      "arn:aws:s3:::test-bucket*",
      "arn:aws:s3:::test-bucket/*/*"
    ]
  },
  {
    "Sid": "OtherActions",
    "Effect": "Allow",
    "Action": [
      "access-analyzer:ValidatePolicy",
      "lambda:ListFunctions",
      "lambda:GetAccountSettings",
      "iam:ListPolicies",
      "iam:ListRoles",
      "iam:ListAttachedRolePolicies",
      "iam:ListRolePolicies",
      "iam:ListInstanceProfilesForRole",
      "iam:ListRoleTags",
      "s3:ListAllMyBuckets",
      "s3:ListBucket",
      "s3:ListBucketVersions"
    ],
    "Resource": "*"
  }
]
}

```

- After writing the JSON for the policy, click the "Next" button.
- Name the policy and click the "Create policy" button.

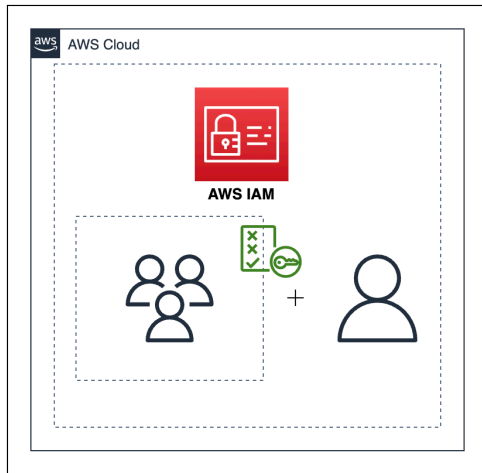
Now, let's create a user group and attach the pre-created policy to provide the required access to that group.

Create an IAM Group and Add a User

A user group is a collection of IAM users. User groups are used to specify permissions to a collection of users. Groups simplify permission management by letting us grant, change, and remove permissions for multiple users simultaneously.

In this task, we'll create a user group and attach an IAM identity-based policy. Then, we'll create an IAM user and add it to our user group, giving it the same set of permissions defined by the policy.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Architecture diagram

Let's create a user group on IAM and attach the pre-created policy to it:

- Search "IAM" on the AWS console and click "IAM" from the search results. This takes us to the IAM dashboard.
- Click "User groups" under "Access management" from the left sidebar.
- Click the "Create group" button to create a new user group.
- Name the group `IAMLabGroup`.
- Scroll down to the "Attach permissions policies - Optional" section and search for the `IAMLabPolicy` policy using the search bar.
- Select the `IAMLabPolicy` policy.
- Now search for and select the `IAMUserChangePassword` policy.
- Click the "Create user group" button.

With that, we've successfully created a user group with the `IAMLabPolicy` and `IAMUserChangePassword` policies attached. Any users added to this group will have the same policy attached to them.

Now, let's create an IAM user and add it to the `IAMLabGroup` group by following the given steps:

- Click "Users" under "Access management" from the left sidebar. This takes us to a list of all available users in our account.
- Click the "Create user" button to create a new user.
- Name the user `IAMLabUser`.
- Check the "Provide user access to the AWS Management Console - optional" option.
 - For "Console password", we can either select the "Autogenerated password" option or the "Custom password" option and set a password of our choice. We'll select the "Autogenerated password" option.
 - Uncheck the "Users must create a new password at next sign-in (recommended)." option and click the "Next" button.
- On the "Set permissions" page, select the "Add user to group" option from the "Permissions options" section.
- Select `IAMLabGroup` in the "User groups" section and click the "Next" button.
- On the "Review and create" page, click the "Create user" button.
- We'll now see the "Console sign-in details." Copy the "Console sign-in URL", "User name", and "Console password", and keep them safe or download the credentials by clicking the "Download .csv file" button.
- Click the "Return to users list" button.

We've now created a new user and added it to the `IAMLabGroup` group. Any users within this group will automatically have the `IAMLabPolicy` policy attached to them.

Now, let's log in as the new user we've just created. Follow the given steps to proceed:

- Open a new incognito window and use that to log in as `IAMLabUser` without logging out from your current AWS session. We are using a new incognito window because we can't have multiple AWS sessions running on the same browser.
- In the incognito window, navigate to the "Console sign-in URL" we copied in the previous step. This takes us to the AWS console sign-in page with the account ID already entered.

- Enter `IAMLabUser` for “IAM user name.”
- Enter the “Console password” that we copied in the last step for “Password.”
- Click the “Sign in” button to log in to the console.

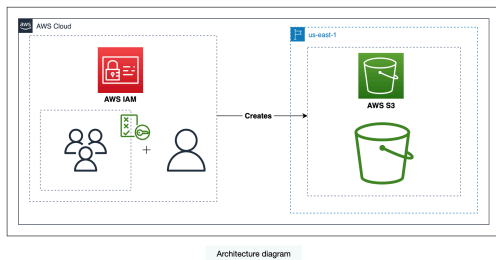
We’ve successfully signed in as the `IAMLabUser` user. We’ll set up the rest of the infrastructure using this newly created user.

- If the IAM user’s account is not already set to `us-east-1`, update the region to `us-east-1`.

Create an S3 Bucket and Upload Files

AWS S3 is a storage service that allows us to store and manage data efficiently. It also allows us to specify access privileges as per our requirements. In this task, we’ll create an S3 bucket and upload a file to the bucket. We’ll later attach a resource-based policy to it to enable public access.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Follow the given steps to create an S3 bucket:

Note: This task is to be done by the IAM user’s account.

- Search “S3” on the AWS console and click “S3” from the search results. This takes us to the S3 dashboard.
- Click the “Create bucket” button.
- Name the bucket `test-bucket<random_text>`, replacing `<random_text>` with a unique text of your choice. Ensure that the bucket name is globally unique across all AWS accounts.

Note: The S3 bucket name should be globally unique because a bucket can be used as a publicly accessible URL.

- In the “Block Public Access settings for this bucket” section, uncheck the “Block all public access” option.
- Acknowledge the public access warning by checking it.
- Scroll down to the end of the page and click the “Create bucket” button. We’ll now be redirected to the S3 Buckets page.

We’ve successfully created an S3 bucket. Now, we’ll upload a file into it by following the given steps:

- Search for the bucket we just created using the search bar in the “Buckets” section.
- Click the bucket name. We’ll be redirected to the bucket page.
- Click the “Upload” button in the “Objects” section. We’ll be redirected to the upload page.
- Download the HTML file given below. We’ll upload this file to the bucket.

```
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
  </header>
  <main>
    <p>This is the main content of my website.</p>
  </main>
</body>
</html>
```

- Click the “Add files” button in the “Files and folders” section.
- Select the `index.html` file to upload and click the “Upload” button at the end of the page.
- After the file is successfully uploaded, we’ll see a success message. Click the “Close” button.

With that, we’ve successfully uploaded a file to our S3 bucket. However, this file isn’t publicly accessible at this point. Let’s verify this by accessing the file using the public URL.

- Select the file we just uploaded from the “Objects” section and copy the URL listed below “Object URL.” This is the publicly accessible URL for the file.
- Open a new browser tab and paste the URL in the search bar. We’ll see an access denied error. This is because the bucket contents are not publicly accessible at this point.

The next step is to make the contents of our S3 bucket publicly accessible. We can do that by creating a resource-based policy and attaching it to our bucket to specify who can access its contents.

Add a Resource-Based Policy

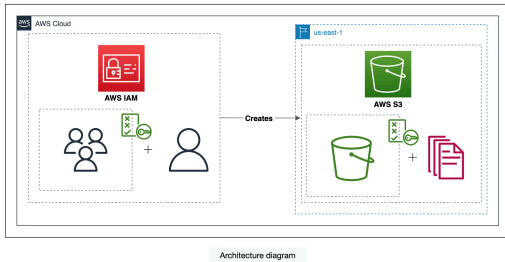
Resource-based policies control access to AWS resources. We can attach these policies to AWS resources, such as S3 buckets, Lambda functions, and others, to specify who can access these resources and the actions they can perform on them.

Similar to identity-based policies, resource-based policies have the same structure with one exception:

- **Effect:** This specifies if the associated actions are allowed or denied. Its value can either be **Allow** or **Deny**.
- **Action:** This contains a list of allowed or denied actions.
- **Resource:** This contains a list of resources for the specified actions.
- **Principal:** This specifies the principal (such as an IAM user, roles, and others) who is allowed (or denied) access to the resource.
- **Condition:** This can be one or a list of additional restrictions regarding when a policy can be in effect.

We can see the inclusion of the **Principal** component for the resource-based policies. The identity-based policies do not contain the **Principal** element as they are attached separately to IAM identities.

The architecture diagram for the infrastructure provisioned in this task would be similar to the one shown in the figure below:



Follow the given steps to create and attach a resource-based policy to the S3 bucket:

Note: This task is to be done by the IAMLabUser's account.

- Go back to the AWS console of the **IAMLabUser** user and navigate to the **test-bucket<appended characters>** page.
- Click the "Permissions" tab on the bucket page.
- Click the "Edit" button in the "Bucket policy" section.
- Copy the policy from the code widget below and paste it into the policy editor window.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Action": "s3:GetObject",
      "Effect": "Allow",
      "Resource": "<ARN>/*",
      "Principal": "*"
    }
  ]
}
```

- Replace **<ARN>** in **line 8** with the ARN of our bucket provided above the editor.
- Click the "Save changes" button.
- Let's get an overview of the bucket policy given above:
- **Line 6:** This is the **Action** we want to permit the specified **Principal** to perform.
- **Line 7:** The **Effect** attribute is set to **Allow** to indicate that we want to permit the specified **Action** to be performed by the given **Principal**.
- **Line 8:** This is the Amazon Resource Name (ARN) of our S3 bucket. Notice the **/*** at the end. This indicates that we have allowed the **s3:GetObject** action to be performed on all bucket objects.
- **Line 9:** The **Principal** attribute is set to *****, since we want to enable public access for everyone.

Now that we've attached a resource-based policy to our S3 bucket to allow public access, we can go ahead and verify if we can access the file using its public URL now. Follow the given steps:

- Click the "Objects" tab.
- Select the file we uploaded from the "Objects" section and copy the URL listed below "Object URL."
- Open a new browser tab and paste the URL in the search bar. This time, we'll see the file contents in the browser window.

We've successfully allowed public access to our S3 bucket contents.

There is another way for us to make the S3 bucket objects accessible to the requesting entity. We'll explore that in the next module. However, to demonstrate this method, we need to delete the existing S3 bucket policy, as it currently makes **index.html** accessible to all requesting entities. Follow these steps to delete the bucket policy:

- Head back to the "Permissions" tab of the bucket and click the "Delete" button in the "Bucket policy" section.
- Enter **delete** in the pop-up window and click the "Delete" button to remove the bucket policy.

You'll see a success message and your policy will be deleted.

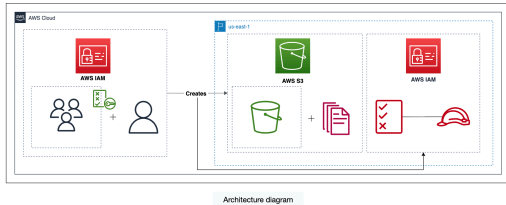
You'll see a success message and your policy will be deleted.

Create an IAM Role

AWS IAM roles are AWS identities with an identity-based policy specifying their access. So far, this is similar to IAM users. However, what makes roles unique is that instead of being directly associated with someone, anyone can assume a role, giving them temporary credentials to access resources based on the policies attached to the role.

Different entities, including IAM users and AWS services, can assume a role. Therefore, a role has an associated trust policy (in addition to the permission policy), which specifies the principals trusted to assume the role. In the context of the use case in this lab, the Lambda function needs to access the S3 bucket and therefore needs permissions. This can be handled by creating a role and allowing lambda to assume the role.

In this task, we'll create an IAM role for the Lambda function. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Follow the given steps to create an IAM role:

Note: This task is to be done by the IAM user's account.

- Search for "IAM" and select "IAM" from the search results. This takes us to the IAM dashboard.
- Click "Roles" under the "Access management" heading in the left menu-bar.
- Click the "Create role" button.
 - For "Trusted entity type", select "AWS service."
 - For "Use case", select "Lambda."
 - Click the "Next" button.
- On the "Add permissions" page, search for and select the "AmazonS3ReadOnlyAccess" policy. This is the policy that will allow our Lambda function to read our S3 bucket objects.
- Search for and select another policy, "AWSLambdaBasicExecutionRole," under the "Permissions policies" section. This policy will allow our Lambda function to write logs in the CloudWatch Logs.
- Click the "Next" button.
- Name the role `TestLambdaRole`.
- Scroll to the end of the page and click the "Create role" button.

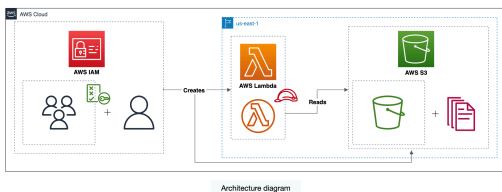
We'll see a confirmation message upon the successful creation of the IAM role.

Next, let's create a Lambda function.

Create a Lambda Function

AWS Lambda functions allow us to execute code without maintaining or deploying servers. They can be considered a serverless design core component and only charge consumers for the compute time used. Lambda supports various programming languages, including Python, Java, Go, and Ruby.

In this task, we'll create a Lambda function and attach an IAM role to enable the function to access S3 objects. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Follow the given steps to create a Lambda function:

Note: This task is to be done by the IAM user's account.

- Search for "Lambda" and select "Lambda" from the search results. This takes us to the Lambda dashboard.
- Click the "Create function" button.
- On the "Create function" page, select "Author from scratch."
- Under the "Basic information" section, enter the following information:
 - For "Function name", enter `TestLambdaFunction`.

- For “Runtime”, choose “Python 3.14” from the drop-down list.
- Click the “Additional settings” in the “Custom settings” section to expand it.
- Click the “Toggle” button from the “Custom execution role” card in the “General” section. It will open the “Configure custom execution role” menu on the right side.
- Select the `TestLambdaRole` role from the “Execution role” drop-down list and click the “Save” button.
- Leave the remaining settings as they are and click the “Create function” button at the end of the page.

We’ve successfully created a Lambda function and attached an execution role to it. It is now time to update the function code to read a file from the S3 bucket. For that, under the “Code” tab on the Lambda function page, replace the code in the `lambda_function.py` file with the code given in the widget below. Make the following changes to the code:

- Replace the value of the `bucket_name` variable on **line 8** with the name of the S3 bucket you created earlier.
- Replace the value of the `file_name` variable on **line 9** with the name of the file you uploaded earlier.

```
import json
import boto3

def lambda_handler(event, context):
    try:
        s3_client = boto3.client('s3')

        bucket_name = '<BUCKET_NAME>'
        file_name = '<FILE_NAME>'

        response = s3_client.get_object(Bucket=bucket_name, Key=file_name)
        data = response['Body'].read().decode('utf')

        return str(data)
    except Exception as e:
        error_message = f"An error occurred: {str(e)}"
        return error_message
```

Now that we’ve updated the code of our Lambda function, we can go ahead and deploy and test it by following the given steps:

- Click the “Deploy” button to deploy the code changes.
- Click the “Test” button to begin creating a test event for the Lambda function.
- Click “Create new test event.”
- For “Event Name”, enter `TestLambdaEvent`.
- For “Event sharing settings”, select “Private.”
- Leave the code in the “Event JSON” editor window as it is and click the “Save” button.
- Now click the “Test” button again to run the test event that invokes the Lambda function. You’ll see the contents of the S3 bucket as the response of the test event.

With that, we’ve successfully accessed an S3 bucket object using a Lambda function with an IAM role.

Clean Up

Let’s end this lab by cleaning up all the resources we’ve created in this lab. We’ll use the same accounts to delete the resources which were used to create them.

Resources created by IAMLabUser

We’ll start with the resources created using the account of `IAMLabUser`.

Delete the Lambda function

Let’s start by deleting the Lambda function by following the given steps:

- Head over to the “Lambda > Functions” page and select the function from the functions list.
- Click the “Actions” button and then click “Delete” from the drop-down menu.
- Enter `confirm` in the pop-up window textbox and click the “Delete” button to confirm the deletion.
- You’ll see the success status indicating the successful deletion of the Lambda function. Click the “Close” button to close the pop-up window.

Delete the S3 bucket

Before we delete our S3 bucket, we must empty it. Follow the given steps to empty our S3 bucket:

- Head over to the “Amazon S3 > Buckets” page and search for your S3 bucket.
- Click the name of the S3 bucket.
- Select `index.html` from the object list and click the “Delete” button.
- Enter `delete` in the textbox on the “Delete objects” page and click the “Delete objects” button.
- After the bucket is successfully emptied, we’ll see a success message. Click the “Close” button.

Now that we've emptied the S3 bucket, we can go ahead and delete the S3 bucket itself. Follow the given steps to delete the S3 bucket:

- Head back to the S3 dashboard and select the bucket from the bucket list.
- Click the "Delete" button.
- Enter the bucket name in the textbox to confirm deletion and click the "Delete bucket" button.

Delete the IAM role

Next, let's delete the IAM role we created for our Lambda function:

- Search for "IAM" and select "IAM" from the search results to head over to the IAM dashboard.
- Click "Roles" under the "Access management" heading in the left menu bar.
- Search for `TestLambdaRole` using the search bar and select the role from the search results.
- Click the "Delete" button.
- Type `TestLambdaRole` in the pop-up window and click the "Delete" button to confirm the deletion.

Now that we've deleted the Lambda function, the IAM role, the S3 bucket, and its contents, we can sign out from our `IAMLabUser` account by clicking the username in the top-right corner and then clicking the "Sign out" button.

Resources created by the original account

Go back to the original browser window from where we started off and clean up the IAM user, group, and group policy.

Delete the IAM user

Follow the given steps to delete the IAM user:

- Close the private window and return to the "IAM > Users" page.
- Select `IAMLabUser` from the list of users and click the "Delete" button.
- Enter `confirm` in the confirmation pop-up and click the "Delete user" button to permanently delete the user.

Delete the user group

Now, let's delete the user group as well by following the given steps:

- Navigate to the "IAM > User groups" page and select the `IAMLabGroup` from the list of user groups.
- Click the "Delete" button.
- Enter `IAMLabGroup` in the confirmation pop-up and click the "Delete" button to permanently delete the user group.

Congratulations! You've successfully completed this Cloud Lab!